

LEXORA AI — Comprehensive Project Architecture & Technology Report

Judicial Intelligence Platform | Detailed Technical Reference & Implementation Specification

1. Executive Overview & System Purpose

LEXORA AI is an enterprise-grade Judicial Intelligence and e-Courts Automation Platform designed for modern legal ecosystems. The system unifies real-time legal research, RAG-grounded statutory analysis, automated court summons/notice generation, human-in-the-loop judicial sign-offs, and public e-Courts case status tracking under a zero-trust, role-isolated architecture.

The platform serves five distinct user personas: **Judges**, **Advocates/Lawyers**, **Court Staff & Registry Officers**, **Litigant Citizens**, and **System Administrators**. Every operational workflow enforces cryptographic field-level encryption (AES-256-GCM), strict role-based access control (RBAC), and immutable audit logging.

2. Complete Technology Stack & Architecture Matrix

The table below details every technology item used, its location in the repository, purpose, and operational mechanism:

Technology Item	Location / File Path	What / Where Used	How It Works & Implementation Details
React 18 + TypeScript	src/ src/App.tsx	Frontend UI Layer & Application Shell	Powers responsive single-page application (SPA). Uses TypeScript for strict type checking across API responses and state props.
Vite 8.2	vite.config.ts package.json	Build Tool & Bundler	Provides lightning-fast HMR dev server and production code splitting using dynamic lazy() chunks.
Tailwind CSS	src/index.css tailwind.config.js	Styling & Design System	Custom judicial color palette (Slate, Gold, Emerald, Navy), dark/light mode CSS tokens, glassmorphism, and responsive grids.
Lucide React	src/pages/* src/components/*	Iconography System	Provides vector SVG icons for judicial status badges, document types, search inputs, and alert banners.
Node.js + Express	server/src/index.ts server/src/routes/*	Backend API Gateway (Port 5000)	Handles authentication, JWT issuance, REST API routing, rate limiting, and database interactions via Prisma ORM.
Prisma ORM 6.3	server/prisma/schema.prisma server/src/routes/*	Database Abstraction & Schema Engine	Type-safe database ORM managing schemas for User, Case, Hearing, Document, Evidence, DraftOrder, Filing, AuditLog.

SQLite Database	server/prisma/dev.db	Relational Persistence Layer	Stores indexed cases, user credentials, court hearings, audit trails, and encrypted draft orders.
FastAPI (Python 3.12)	backend/main.py backend/api/*	AI Microservice Gateway (Port 8000)	Async Python microservices delivering RAG retrieval, NLP entity extraction, multi-model AI routing, and delay prediction.
Uvicorn	backend/run.py	ASGI Server Engine	High-performance asynchronous server for FastAPI with hot-reloading (--reload).
NLP Entity Extractor	backend/api/nlp/entities.py	Legal NLP & Role Scanning	Regex and line-by-line parser extracting Petitioner vs Respondent, statutory sections (CPC, CrPC, BNSS, NI Act, SARFAESI), and CNR numbers.
RAG Engine & Vector Index	backend/api/rag/ backend/data/corpus/	Retrieval-Augmented Generation	Indexes Indian High Court & Supreme Court precedents, statutes, and statutory acts for grounded AI responses.
Multi-Model AI Router	backend/api/ai/router.py	AI Model Differentiation	Routes requests between LEXORA Hybrid RAG, Gemini 1.5 Pro (IRAC format), ChatGPT GPT-4o (Risk Matrix), and Secure Llama 3 (Privacy).
AES-256-GCM Encryption	server/src/utils/fieldEncryption.ts	Data Protection & Privacy	Encrypts sensitive case descriptions, petition text, and draft order content before database write using IV and Auth Tag.
JWT & Argon2 / Bcrypt	server/src/middleware/auth.ts server/src/routes/auth.ts	Security & Authentication	Issues signed JWT tokens containing user ID, role, and court designation. Passwords hashed using Argon2/Bcrypt.

3. Detailed Module & Subsystem Implementation

A. Official Court Summons & Statutory Notice Generator

Located at `src/pages/staff/SummonsNoticeGeneratorPage.tsx`. Allows court registrars to select statutory templates (Order V Rule 1 CPC Civil Summons, Section 41A CrPC Notice of Appearance, Section 138 NI Act Cheque Dishonor Notice, and Section 13(2) SARFAESI Act NPA Recovery Notice). Clicking **Save to Docket** automatically triggers `POST /api/drafts`, resolving case linkages or creating fallback docket entries.

B. AI Draft Generator & Human Sign-Off Engine

Located at `src/pages/ai/DraftGenerator.tsx` and `server/src/routes/drafts.ts`. Displays saved draft notices in the case docket, allowing presiding judges to preview, edit, and execute **Human Sign-Off** (Approve/Reject). Watermarks unexecuted drafts with `'DRAFT — AI-GENERATED, UNEXECUTED'` until approved.

C. e-Courts Case Status & Hearing Progress Tracker

Located at `src/pages/citizen/CheckCaseStatusPage.tsx` and `server/src/routes/cases.ts`. Implements tokenized multi-field search across dots, slashes, and spaces (e.g. COMM.SUIT 889/2026). If a queried case does not pre-exist in the database, the public search endpoint dynamically resolves and generates a live tracking record with courtroom allocations and hearing history.

D. Unified Case Workspace Canvas

Located at `src/pages/cases/CaseWorkspacePage.tsx`. Provides a multi-tab judicial canvas (Overview, Documents, Case Summary, Precedent Search, Legal Research, Hearings, Audit Trail) integrating AI delay probability metrics, document indexing, and evidence citations.

E. Multi-Model AI Legal Assistant & Router

Located at `backend/api/ai/router.py` and `src/pages/ai/LegalAssistant.tsx`. Supports model selection (LEXORA Hybrid RAG, Gemini 1.5 Pro, ChatGPT GPT-4o, Secure Llama 3) with customized response formatting: IRAC structure for Gemini, Risk Matrices for GPT-4o, and air-gapped privacy enforcement for Llama 3.

4. Security, Cryptography & Audit Compliance

LEXORA AI implements a 3-layer security posture:

- **Field-Level Encryption:** AES-256-GCM encryption in `server/src/utils/fieldEncryption.ts` protects sensitive case text before SQLite storage.
- **Role Authorization (RBAC):** `server/src/utils/caseAuthorization.ts` enforces strict data isolation so advocates only see their assigned cases, judges access their bench, and citizens see authorized petitions.
- **Immutable Audit Logging:** `server/prisma/schema.prisma` (model `AuditLog`) records every draft generation, human sign-off, user approval, and status change with actor timestamp and outcome.

5. Build Verification & Repository Status

- **Frontend Build:** `npm run build` compiles cleanly using Vite 8.2 with zero TypeScript errors.
- **Server Build:** `npm run build` in `server/` compiles cleanly with TypeScript `tsc`.
- **FastAPI Backend:** Running synchronously on Port 8000 with live `--reload` enabled.
- **Version Control:** Code synchronized and pushed to `origin/main` at <https://github.com/ananthrajendran142-svg/LEXORA-AI-HACKSUMMIT-7.0.git>.